Programación Orientada a Objetos

Clase 8 Continuación - Introducción al paradigma funcional

PARADIGMAS DE PROGRAMACION

UTN - La Plata

Temario

1) Listas
2) Ejemplos
3) Actividad 2

1) Listas

- Son conjuntos de elementos del mismo tipo, incluso puede haber listas de listas.

Notación:

- Lista vacía [ ]

Ejemplos:
[ ]

- Lista unitaria [X]

[3]

- Lista simple Xs o (X: Xs) X→ cabeza, Xs → cuerpo

[2, 3, 6, 8 ,9]

- Lista de listas Xss o (Xs: Xss) Xs→ cabeza, Xss → cuerpo

[[2], [2, 4], [8, 11]]

1) Listas

Operaciones sobre listas:

- Concatenación
  - [ ] : Xs = Xs : [ ] = Xs
  - (Xs :Ys) : Zs = Xs : (Ys : Zs)
- Longitud
  - #[ ] =0
  - # (Xs : Ys)= #Xs + #Ys
- Head
  - hd(X : Xs)= X hd(Xs: Xss)= Xs
- Tail
  - tl(X : Xs)= Xs tl(Xs:Xss)= Xss

1) Listas

Funciones de alto orden

- map
  - cubo(X):=X*X*X
  - cuboL(X:Xs):= map (cubo(X)) Xs

?cuboL([2,4,3])

[8,64,27]

- filter

listaPares(X:Xs):= filter(par(X)) Xs

where par(X) :=true, if X mod 2=0

:= false, otherwise

?listaPares[2,3,8,9]

[2,8]

1) Listas

- take
  - take(0, [21 ,3 ,19]) → [ ]
  - take(2, [21, 3, 19]) → [21, 3]

- takewhile
  - takewhile(par(X), [24, 6,3,18,25]) → [24 6]

Toma los primeros elementos consecutivos que cumplen con la condición especificada.

2) Ejemplos

- Definir y dar el tipo de una función para:

  a) Sumar todos los elementos de una lista

```
sumar::

sumar(X:Xs):=

sumar([]):=
```

2) Ejemplos

- Definir y dar el tipo de una función para:

  a) Sumar todos los elementos de una lista

```
sumar:: [num] → num

sumar(X:Xs):= X + sumar(Xs)

sumar([]):=0
```

2) Ejemplos

- Definir y dar el tipo de una función para:

  b) Contar las veces que aparece un numero N en la lista

```
contar::

contar(X:Xs,N):=

contar([],N):=
```

2) Ejemplos

- Definir y dar el tipo de una función para:

  b) Contar las veces que aparece un numero N en la lista

  contar:: [num],num → num

  contar(X:Xs,N):= 1 + contar(Xs,N), if X=N

  := contar(Xs,N), otherwise

  contar([],N):=0

2) Ejemplos

- Definir y dar el tipo de una función para:

  c) Determinar si en una lista de booleanos hay algún valor verdadero

```
algunv::

algunv(X:Xs):=


algunv([]):=
```

2) Ejemplos

- Definir y dar el tipo de una función para:

  c) Determinar si en una lista de booleanos hay algún valor verdadero

```
algunv:: [bool] → bool

algunv(X:Xs):= true, if X=true

                := algunv(Xs), otherwise

algunv([]):= false
```

2) Ejemplos

- Definir y dar el tipo de una función para:

  d) Dada una lista de listas retornar una lista con la longitud de cada sublista

```
longLis::
longLis([]):=
longLis(Xs:Xss):=
```

2) Ejemplos

- Definir y dar le tipo de una función para:

  d) Dada una lista de listas retornar una lista con la longitud de cada sublista

  longLis:: [[α]] → [num] [[1,2], [], [6,4,12,9,0]] retorna [2,0,5]

  longLis([ ]):=[ ]

  longLis(Xs:Xss):= #Xs : longLis(Xss)

2) Ejemplos

- Definir y dar el tipo de una función para:

  e) Dada una lista de listas, un carácter C y un número K, retornar las listas en las que el carácter C aparece K veces.

```
estaCenLis::
estaCenLis([],C,K):=
estaCenLis(Xs:Xss,C,K):=
                              :=
veces::
veces([],C):=
veces(X:Xs,C):=
                    :=
```

2) Ejemplos

- Definir y dar el tipo de una función para:

  e) Dada una lista de listas, un carácter C y un número K, retornar las listas en las que el carácter C aparece K veces.

```
estaCenLis:: [[ char]], char, num → [[char]]
estaCenLis([],C,K):=[]
estaCenLis(Xs:Xss,C,K):= Xs:  estaCenLis(Xss,C,K), if veces(Xs,C)=K
                          := estaCenLis(Xss,C,K), otherwise

veces::[char],char → num
veces([],C):=0
veces(X:Xs,C):= 1 + veces(Xs,C), if X=C
             := veces(Xs,C), otherwise
```

2) Actividad 2

- Definir y dar el tipo de una función para:

  1) Determinar si en una lista de booleanos todos los elementos son verdaderos.

  2) Elevar al cuadrado todos los elementos de una lista.

  3) Dada una lista de listas y un número K, retornar las sublistas cuyos elementos sumen K.

Ej. L=[[6,14], [7,10,3], [15,20,3,50]) K=20

Retorna [[6,14], [7,10,3]]



Resultados de la Actividad 2

en las próximas diapositivas

3) Actividad 2

- Definir y dar el tipo de una función para:

  1) Determinar si en una lista de booleanos todos los elementos son verdaderos.

  ```
  todosv:: [bool] → bool
  todosv(X:Xs):= false, if X=false
                  := todosv(Xs), otherwise
  todosv([]):= true
  ```

- o
  ```
  todosv:: [bool] → bool
  todosv(Xs):= true     if #(filter (X=true) Xs )= #Xs
               := false    otherwise
  ```

8) Actividad 2

- Definir y dar el tipo de una función para:

  2) Elevar al cuadrado todos los elementos de una lista.

```
cuadrado:: num → num

cuadrado(X):=X*X

cuadradosL:: [num] → [num]

cuadradosL(X:Xs):= map (cuadrado (X)) Xs
```

8) Actividad 2

- Definir y dar el tipo de una función para:

3) Dada una lista de listas y un número K, retornar las sublistas cuyos elementos sumen K.

Ej. L=[[6,14], [7,10,3], [15,20,3,50]) K=20 Retorna [[6,14], [7,10,3]]

```
sumenk:: [[num]],num → [[num]]
sumenk ((Xs:Xss),k):= Xs: sumenk(Xss,k) ,     if suma(Xs)=k
                    := sumenk(Xss,k)  ,       otherwise

suma:: [num] → num
suma(X:Xs):= X + suma(Xs)
suma([]):=0
```